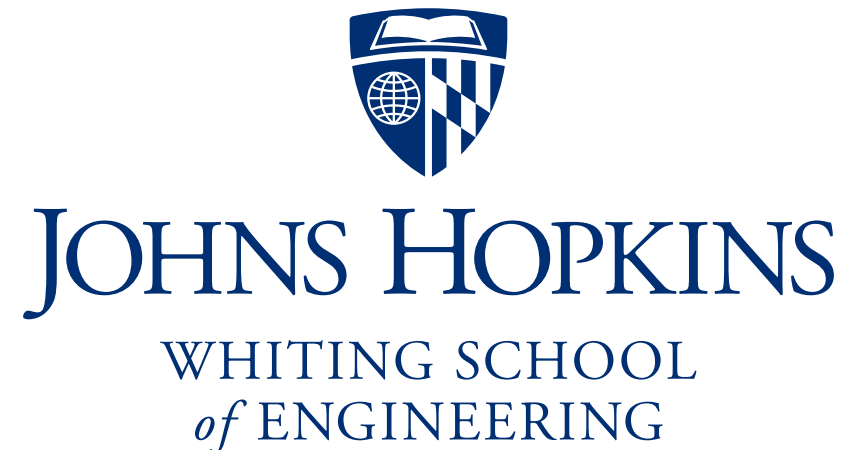


Assembly & shortest common superstring

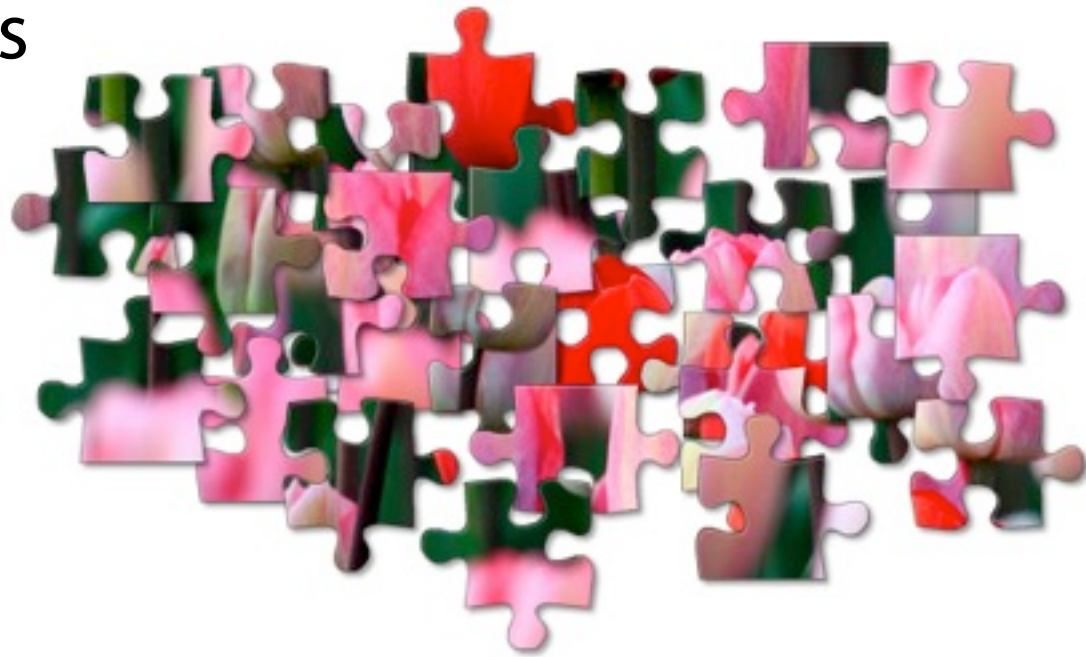
Ben Langmead



You are free to use these slides. If you do, please sign the guestbook (www.langmead-lab.org/teaching-materials), or email me (ben.langmead@gmail.com) and tell me briefly how you're using them. For original Keynote files, email me.

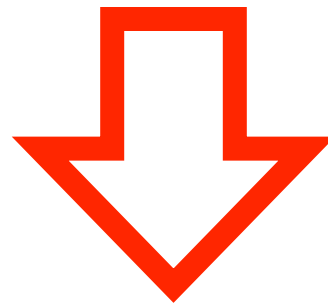
Assembly

Reads



+

Reference genome



Input DNA



How to assemble
puzzle without the
benefit of knowing
what the finished
product looks like?

Assembly

Whole-genome “shotgun” sequencing starts by copying and fragmenting the DNA

(“Shotgun” refers to the random fragmentation of the whole genome; like it was fired from a shotgun)

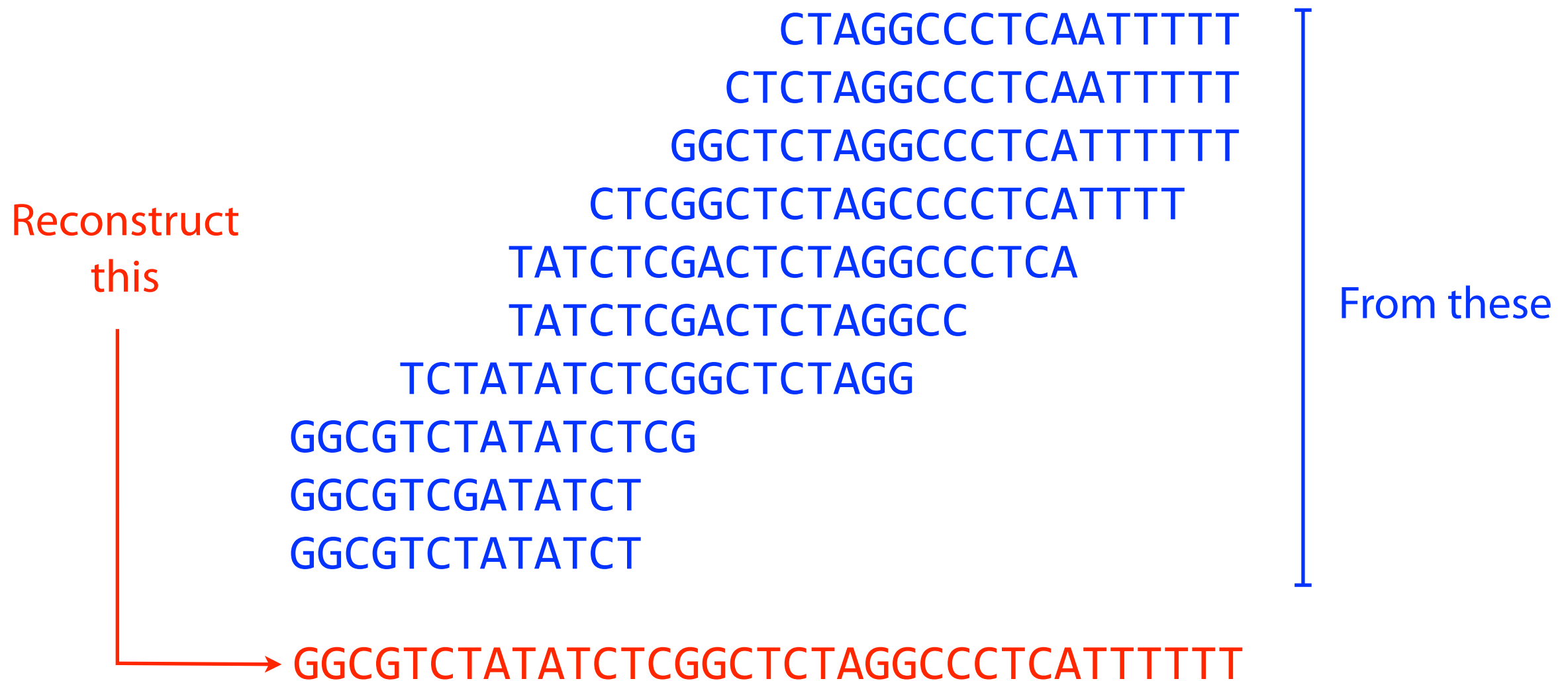
Input: GCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT

Copy: GCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT
GCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT
GCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT
GCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT

Fragment: GCGTCTA TATCTCGG CTCTAGGCCCTC ATTTTTT
GGC GTCTATAT CTCGGCTCTAGGCCCTCA TTTTTT
GCGTC TATATCT CGGCTCTAGGCCCT CATTTTTT
GCGTCTAT ATCTCGGCTCTAG GCCCTCA TTTTTT

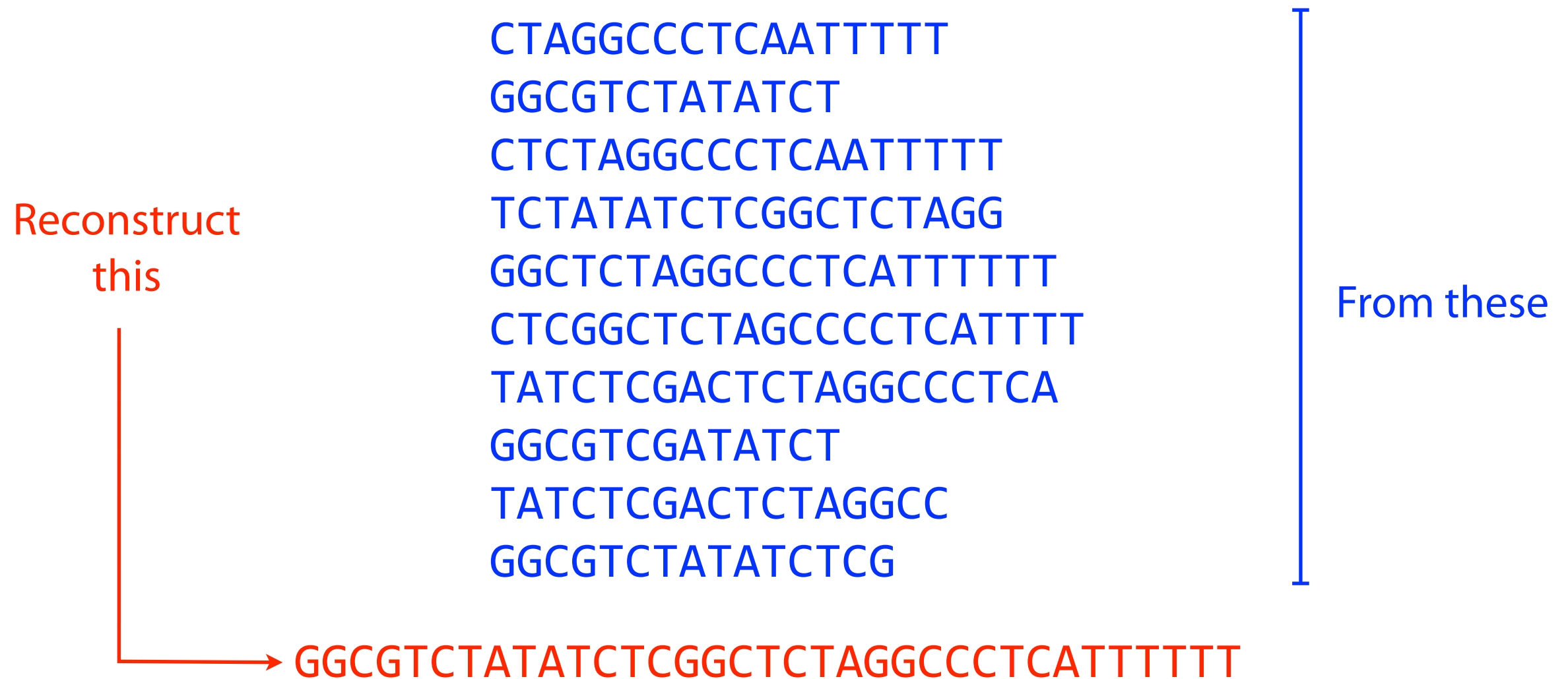
Assembly

Assume sequencing produces such a large # fragments that almost all genome positions are *covered* by many fragments...



Assembly

...but we don't know what came from where



Assembly

Key term: *coverage*. Usually it's short for *average coverage*: the average number of reads covering a position in the genome.

```
          CTAGGCCCTCAATTTT
        CTCTAGGCCCTCAATTTT
      GGCTCTAGGCCCTCATTTTT
    CTCGGCTCTAGCCCCTCATTTT
  TATCTCGACTCTAGGCCCTCA
  TATCTCGACTCTAGGCC
TCTATATCTCGGCTCTAGG
GGCGTCTATATCTCG
GGCGTCGATATCT
GGCGTCTATATCT
GGCGTCTATATCTCGGCTCTAGGCCCTCATTTTTT
```

177 nucleotides

35 nucleotides

$$\text{Average coverage} = 177 / 35 \approx 7x$$

Assembly

Coverage could also refer to the number of reads covering a particular position in the genome:

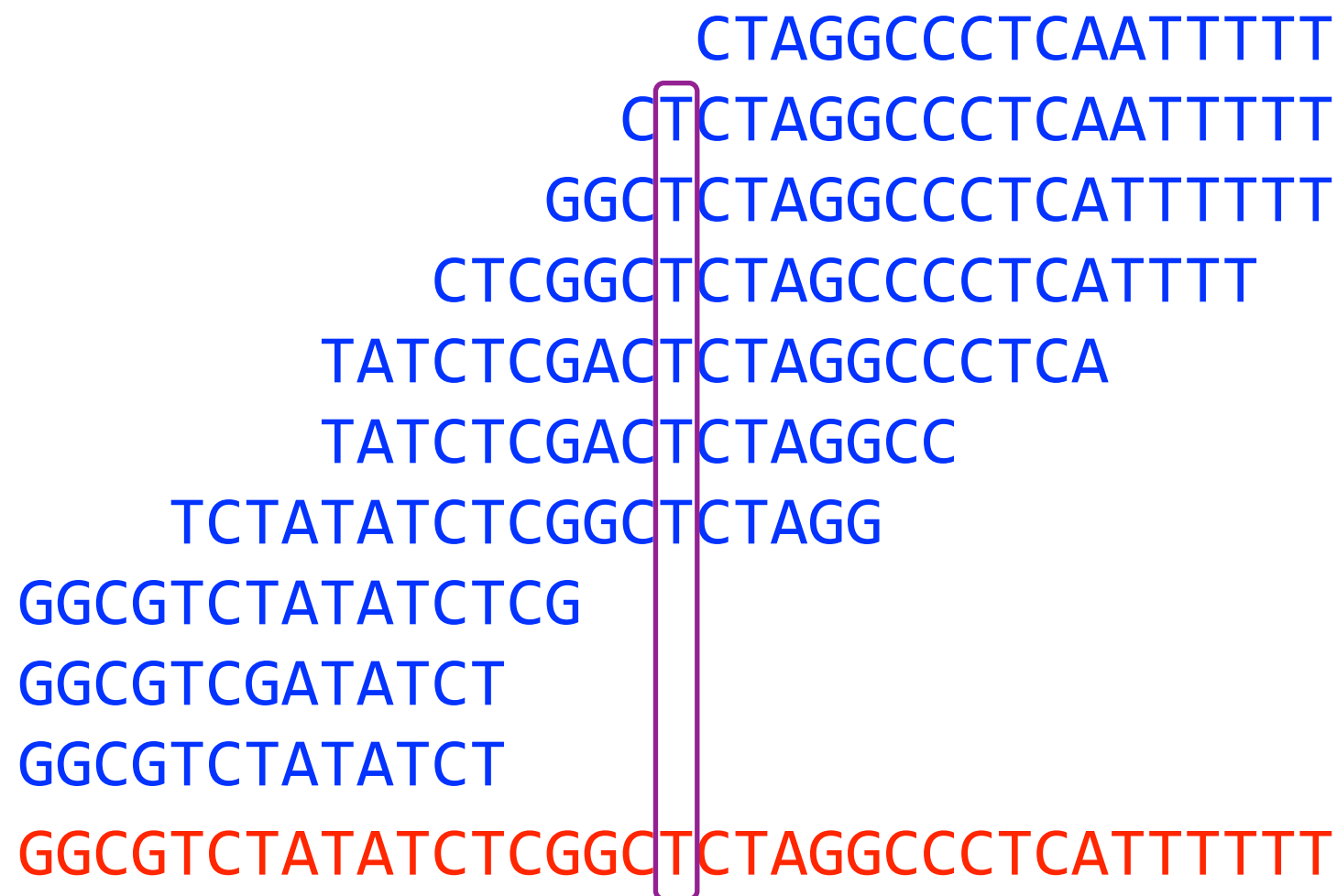


Diagram illustrating sequence alignment and coverage. The reads are aligned as follows:

```
          CTAGGCCCTCAATTTT
        CTCTAGGCCCTCAATTTT
      GGCTCTAGGCCCTCATTTT
    CTCGGCTCTAGCCCCTCATTT
  TATCTCGACTCTAGGCCCTCA
  TATCTCGACTCTAGGCC
TCTATATCTCGGCTCTAGG
GGCGTCTATATCTCG
GGCGTCGATATCT
GGCGTCTATATCT
GGCGTCTATATCTCGGCTCTAGGCCCTCATTTT
```

A vertical purple box highlights the position corresponding to the 10th nucleotide of the bottom-most read (GGCGTCTATATCTCGGCTCTAGGCCCTCATTTT). An arrow points to this position, indicating the coverage count.

Coverage at this position = 6

Assembly

Basic principle: the more similarity there is between the end of one read and the beginning of another...

```
      TATCTCGACTCTAGGCC
      |||||
TCTATATCTCGGCTCTAGG
```

...the more likely they are to have originated from overlapping stretches of the genome:

```
      TATCTCGACTCTAGGCC
      TCTATATCTCGGCTCTAGG
GGCGTCTATATCTCGGCTCTAGGCCCTCATTTTTT
```


Assembly

Say two reads truly originate from overlapping stretches of the genome. Why might there be differences?

```
      TATCTCGACTCTAGGCC
      |||||
TCTATATCTCGGCTCTAGG
      ↑
```

1. Sequencing error
2. Difference between inherited *copies* of a chromosome

E.g. humans are diploid; we have two copies of each chromosome, one from mother, one from father. The copies can differ:

```
Read from Mother:      TATCTCGACTCTAGGCC
                        |||||
```

```
Read from Father: TCTATATCTCGGCTCTAGG
```

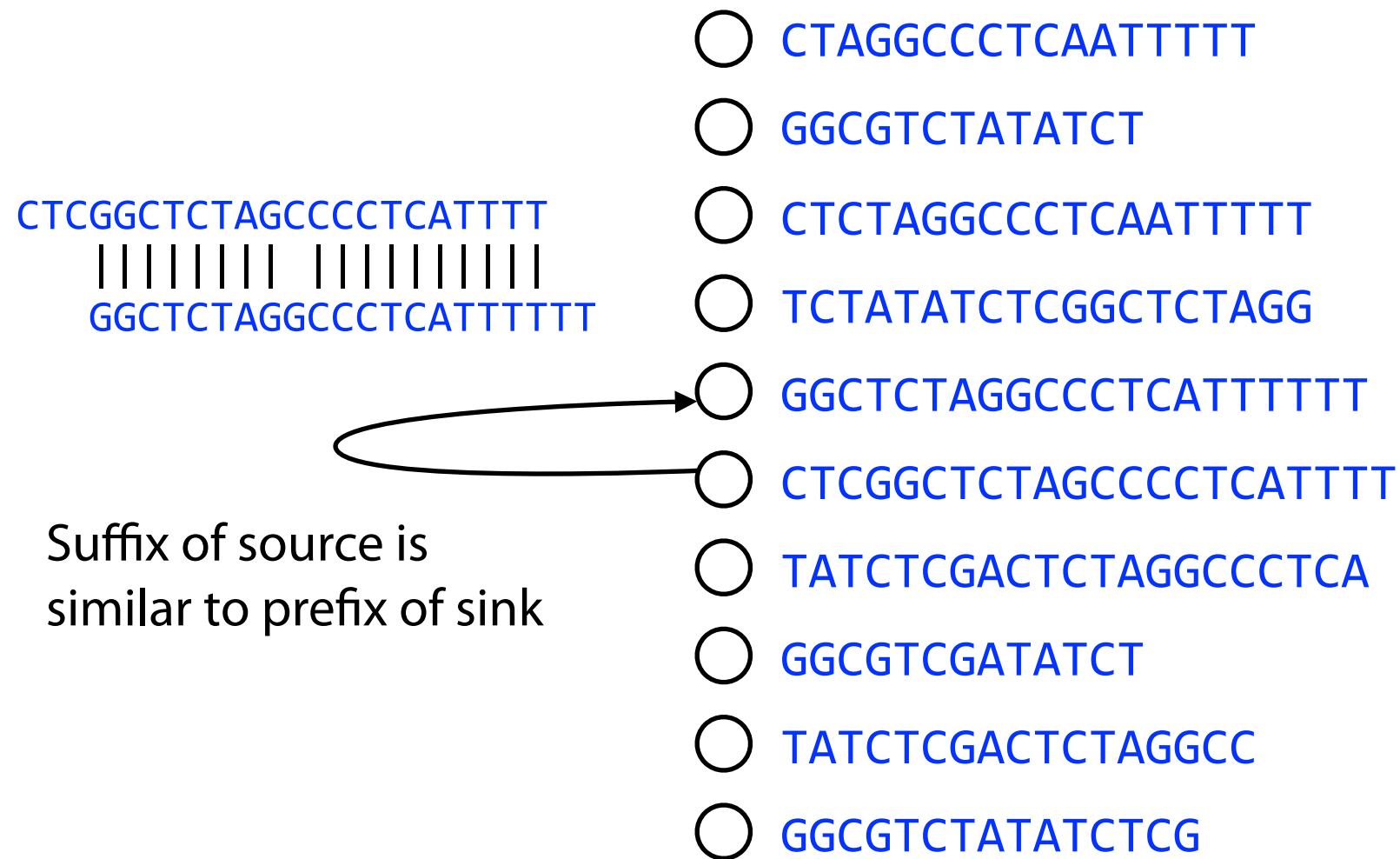
```
Sequence from Mother: TCTATATCTCGACTCTAGGCC
```

```
Sequence from Father: TCTATATCTCGGCTCTAGGCC
```

We'll mostly ignore ploidy, but real tools must consider it

Overlaps

Finding all overlaps is like building a *directed graph* where directed edges connect overlapping nodes (reads)



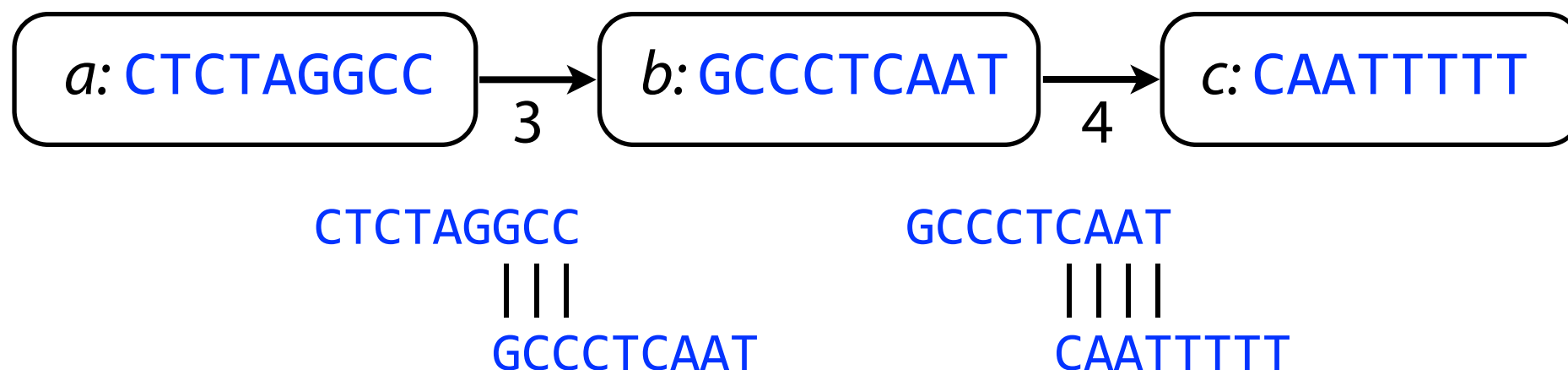
Overlap graph

Below: overlap graph, where an overlap is a suffix/prefix match of at least 3 characters

A vertex is a read, a directed edge is an overlap between suffix of source and prefix of sink

Vertices (reads): { a : CTCTAGGCC, b : GCCCTCAAT, c : CAATTTT }

Edges (overlaps): { (a, b) , (b, c) }



Finding overlaps



How do we build the overlap graph?

What constitutes an overlap?

Assume for now an “overlap” is when a suffix of X of length $\geq l$ exactly matches a prefix of Y , where k is given

Finding overlaps

Overlap: length- l suffix of X matches length- l prefix of Y , where l is given

Simple idea: look in Y for occurrences of length- l suffix of X . Extend matches to the left to confirm whether entire prefix of Y matches.

Say $k = 3$

Look for this in Y ,
going right-to-left

X : CTCTAGGCC
 Y : TAGGCCCTC

X : CTCTAGGCC
 Y : TAGGCCCTC

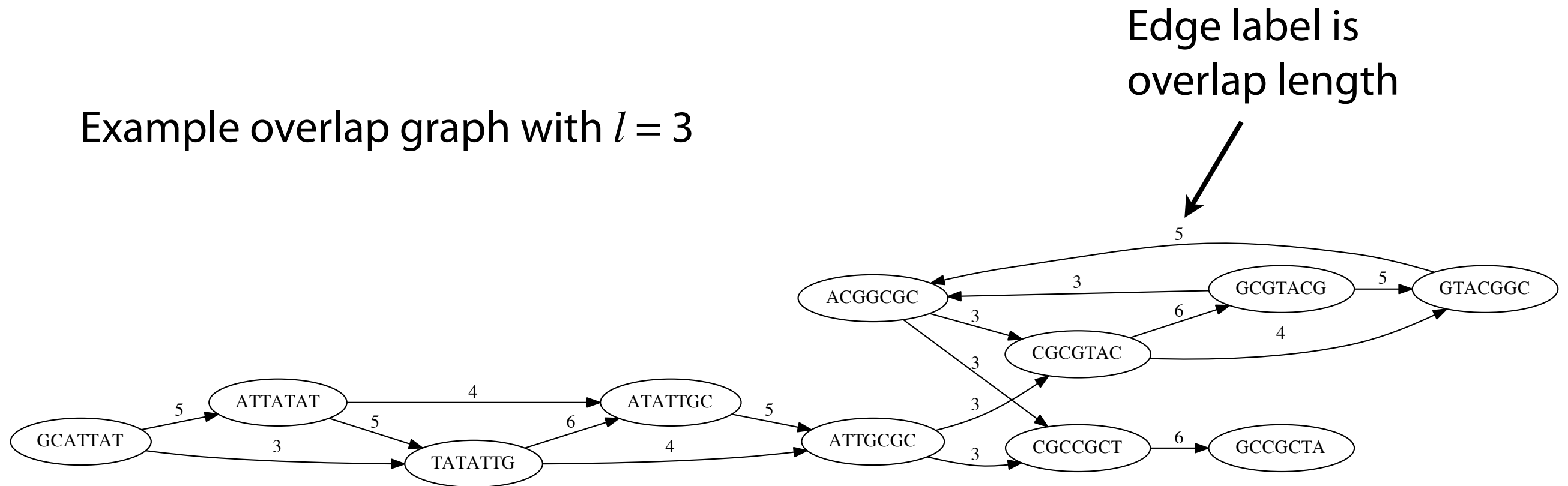
Found it

Extend to left; in this case, we
confirm that a length-6 prefix
of Y matches a suffix of X

X : CTCTAGGCC
 Y : TAGGCCCTC

Finding overlaps

Example overlap graph with $l = 3$



Original string: GCATTATATATTGCGCGTACGGCGCCGCTACA

Formulating the assembly problem

Finding overlaps is important, and we'll return to it, but our ultimate goal is to recreate (assemble) the genome

How do we formulate this problem?

First attempt: the *shortest common superstring* (SCS) problem

Shortest common superstring

Given a collection of strings S , find $SCS(S)$: the shortest string that contains all strings in S as substrings

Without requirement of “shortest,” it’s easy: just concatenate them

Example: S : BAA AAB BBA ABA ABB BBB AAA BAB

Concatenation: BAAAABBBBAABAABBBBBBAAABAB
└────────── 24 ─────────┘

$SCS(S)$: AAABBBBABAA
└── 10 ─┘

AAA
AAB
ABB
BBB
BBA
BAB
ABA
BAA

Shortest common superstring

Can we solve it?

Imagine a modified overlap graph where each edge has cost = - (length of overlap)

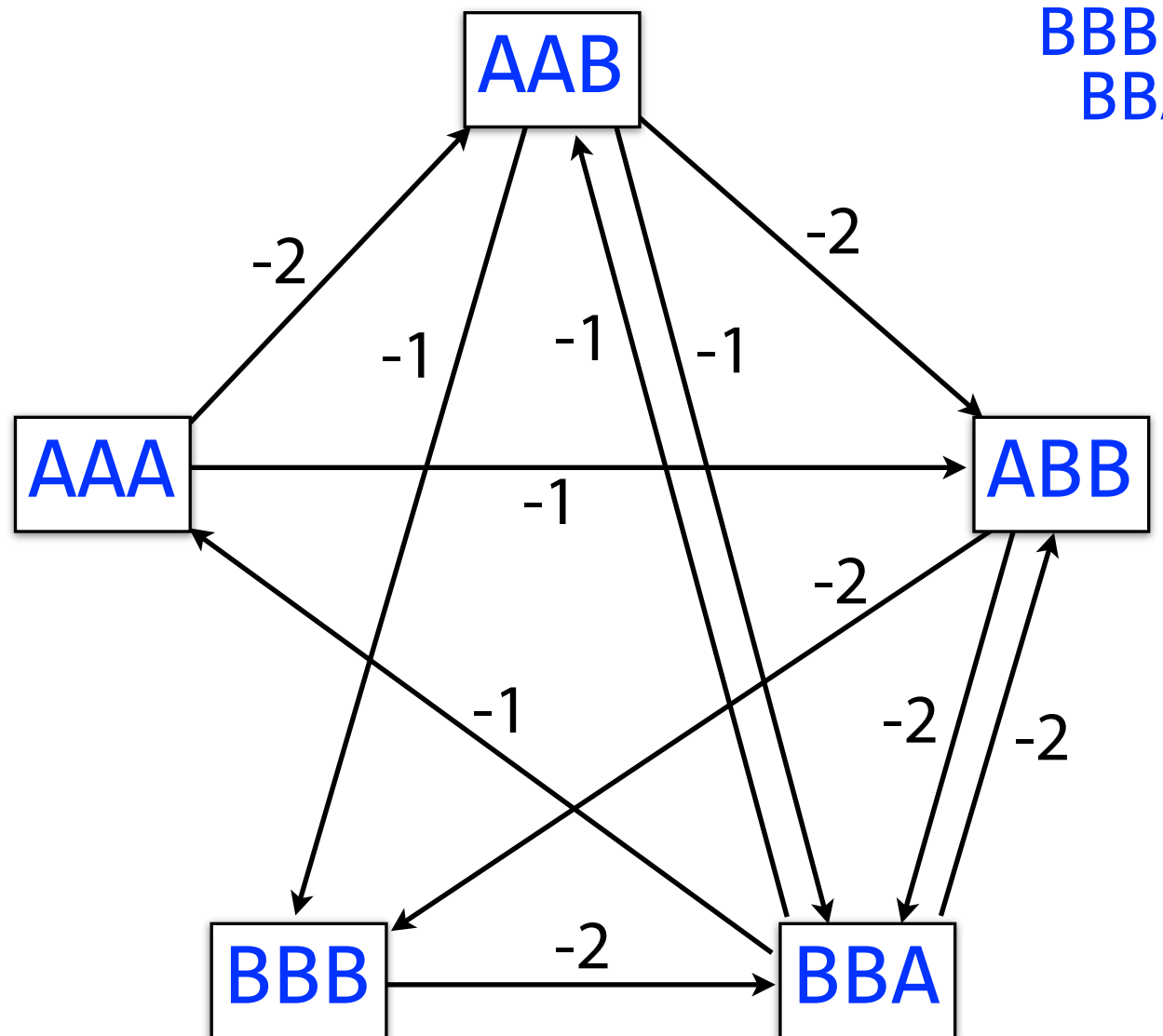
SCS corresponds to a path that visits every node once, minimizing total cost along path

That's the *Traveling Salesman Problem (TSP)*, which is NP-hard!

S: AAA AAB ABB BBB BBA

SCS(S): AAABBBBA

AAA
AAB
ABB
BBB
BBA



Shortest common superstring

Say we disregard edge weights and just look for a path that visits all the nodes exactly once

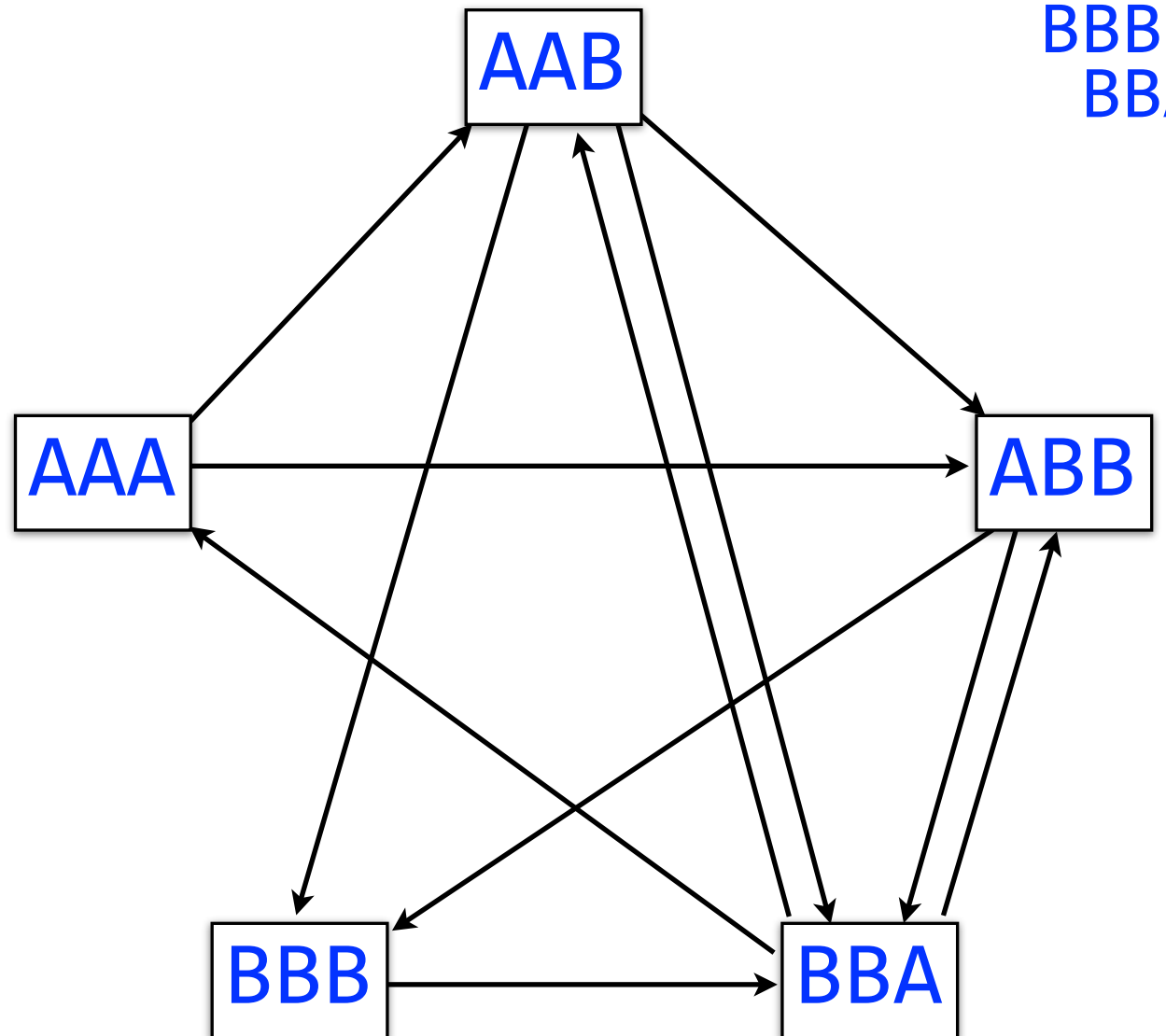
That's the *Hamiltonian Path* problem: NP-complete

Indeed, it's well established that SCS is NP-hard

S: AAA AAB ABB BBB BBA

SCS(S): AAABBBBA

AAA
AAB
ABB
BBB
BBA



Shortest common superstring & friends

Traveling Salesman, Hamiltonian Path, and Shortest Common Superstring are all NP-hard

For refreshers on Traveling Salesman, Hamiltonian Path, NP-hardness and NP-completeness, see Chapters 34 and 35 of “Introduction to Algorithms” by Cormen, Leiserson, Rivest and Stein, or Chapters 8 and 9 of “Algorithms” by Dasgupta, Papadimitriou and Vazirani (free online: <http://www.cs.berkeley.edu/~vazirani/algorithms>)

Shortest common superstring

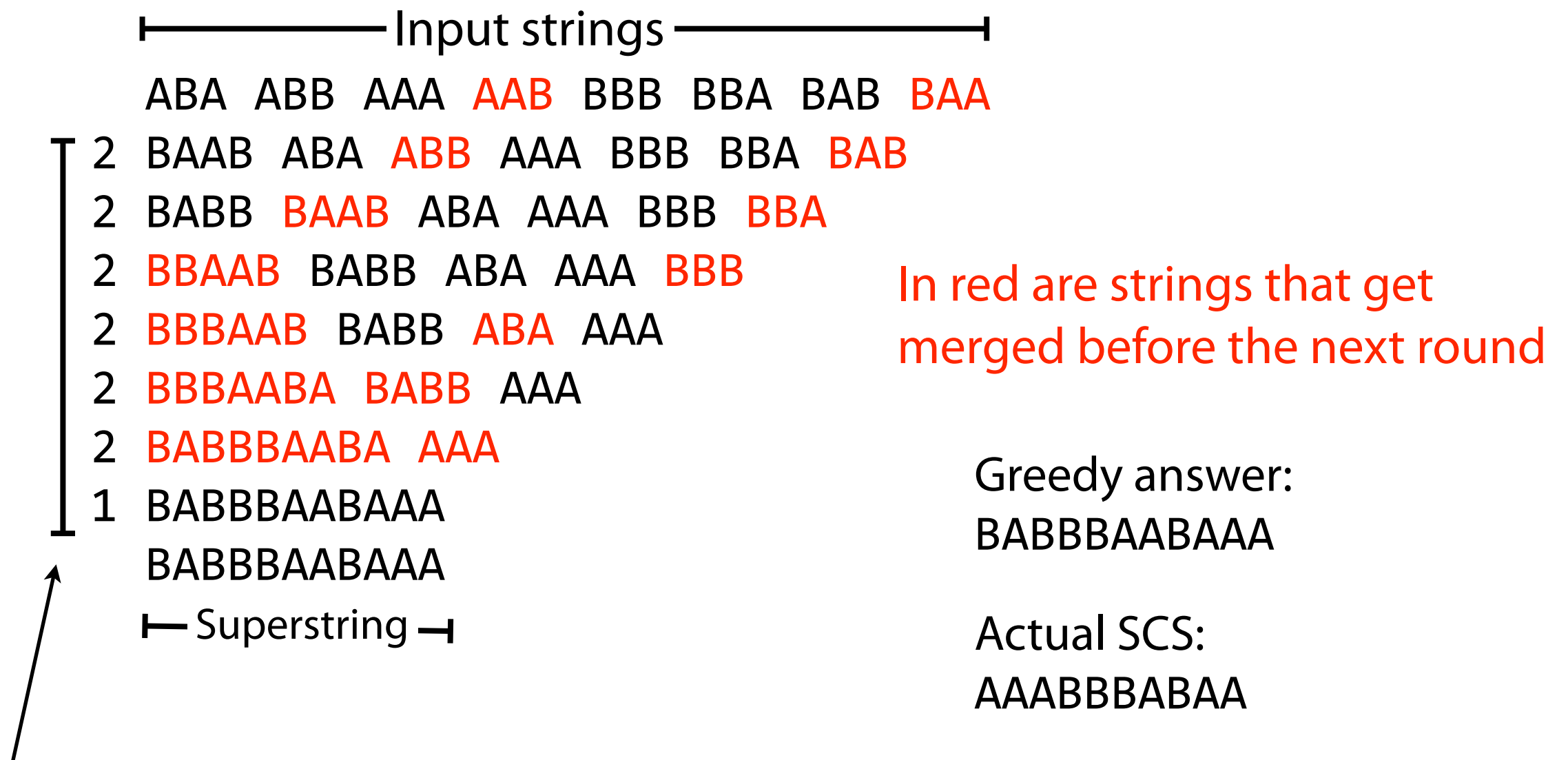
Let's take the hint give up on finding the *shortest possible* superstring

Non-optimal superstrings can be found with a *greedy* algorithm

At each step, the greedy algorithm “greedily” chooses longest remaining overlap, merges its source and sink

Shortest common superstring: greedy

Greedy-SCS algorithm in action ($l = 1$):



Rounds of merging, one merge per line.

Number in first column = length of overlap merged before that round.

Shortest common superstring: greedy

Greedy algorithm is *not* guaranteed to choose overlaps yielding SCS

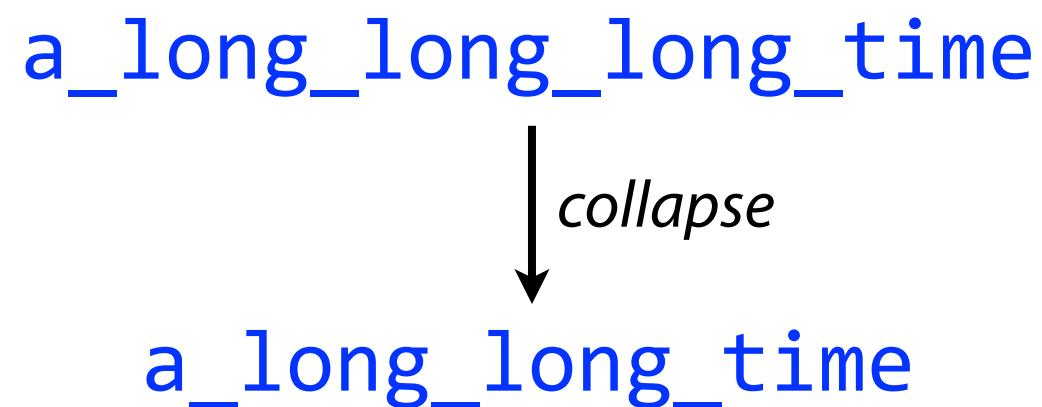
But greedy algorithm is a good *approximation*; i.e. the superstring yielded by the greedy algorithm won't be more than ~ 2.5 times longer than true SCS (see Gusfield 16.17.1)

Repeats

Repeats often foil assembly. They certainly foil SCS, with its “shortest” criterion!

Reads might be too short to “resolve” repetitive sequences. This is why sequencing vendors try to increase read length.

Algorithms that don’t pay attention to repeats (like our greedy SCS algorithm) might *collapse* them



The human genome is ~ 50% repetitive!

Repeats

Basic principle: *repeats foil assembly*

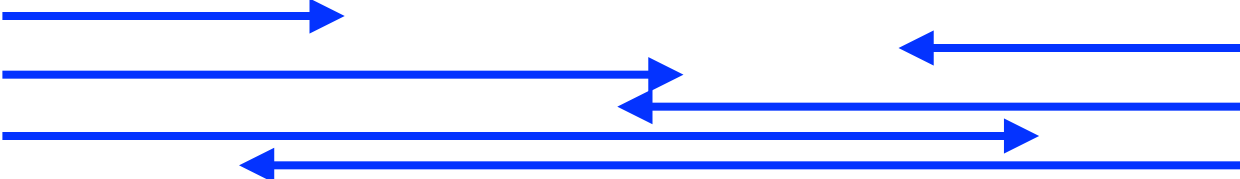
Longer and longer substrings allow us to “anchor” more of the repeat to its non-repetitive context:

swinging_and_the_ringing_of_the_bells_bells_bells_bells_bells



Often we can “walk in” from both sides. When we meet in the middle, the repeat is resolved:

ringing_of_the_bells_bells_bells_bells_bells_to_the_rhythming



Repeats

Basic principle: *repeats foil assembly*

Yet another example using Greedy-SCS:

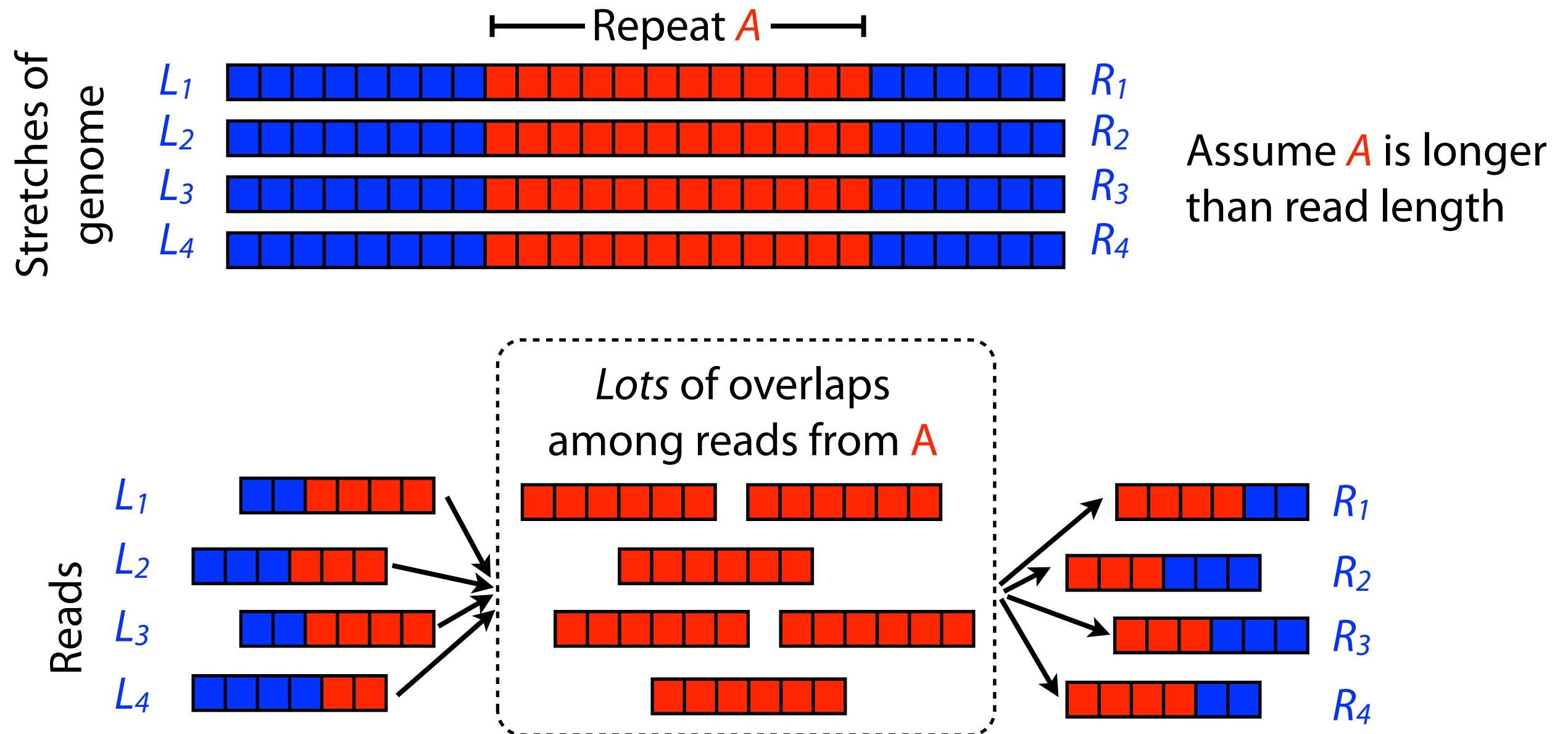
Input: `swinging_and_the_ringing_of_the_bells_bells_bells_bells_bells`

l, k	output
3, 7	<code>swinging_and_the_ringing_of_the_bells_bells</code>
3, 13	<code>swinging_and_the_ringing_of_the_bells_bells_bells</code>
3, 19	<code>swinging_and_the_ringing_of_the_bells_bells_bells_bells_b</code>
3, 25	<code>swinging_and_the_ringing_of_the_bells_bells_bells_bells_bells</code>

longer and longer substrings allow
us to “reach” further into the repeat

Repeats

Picture the portion of the overlap graph involving repeat A



Even if we avoid collapsing copies of A , we can't know which paths *in* correspond to which paths *out*

Shortest common superstring: post mortem

SCS is flawed as a way of formulating the assembly problem

No tractable way to find optimal SCS

Had to use Greedy-SCS. Answers might be too long.

SCS spuriously collapses repetitive sequences

Answers might be too short, by a lot!

Need formulations that are (a) tractable, and (b) handle repeats as gracefully as possible

Remember: repeats foil assembly no matter the algorithm. This is a property of read length and repetitiveness of the genome.